

[CS230] Improving Deep Neural Networks: Mini-batch Gradient Descent

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-4. Neural Networks Basics - *Completed*

Chapter 5. Practical Aspects of Deep Learning (Current Unit)

- 5.1-5.4 Regularization & Data Setup - *Completed*
- 5.5 Input Normalization - *Completed*
- **5.6 Optimization Algorithms 1: Mini-batch Gradient Descent**
 - * Batch vs Stochastic vs Mini-batch
 - * Epoch vs Iteration Definition
 - * Why Powers of 2? (2^n)
 - * Implementation: Shuffle and Partition
- 5.7 Optimization Algorithms 2: Momentum - *Upcoming*

지난 시간 복습 및 연결

지난 강의까지 우리는 과대적합을 막는 '방어 기술 (Regularization)'을 익혔습니다. 이제부터는 모델의 학습 속도를 극한으로 끌어올리는 '가속 기술 (Optimization)'을 다룹니다. 딥러닝의 연료는 데이터입니다. 데이터가 1000만 개라면, 기존 방식 (Batch Gradient Descent)으로는 한 걸음 떼는데 며칠이 걸립니다. 이를 해결하기 위해 데이터를 작은 덩어리로 쪼개서 학습하는 **미니 배치 (Mini-batch)** 기법이 등장했습니다. 이는 현대 딥러닝의 **사실상 표준 (Standard)**입니다.

1 Unit Overview

□ 핵심 목표

이 단원은 대용량 데이터를 효율적으로 학습시키는 **미니 배치 경사 하강법**을 다룹니다.

- **비교:** Batch(전체), Stochastic(1개), Mini-batch(덩어리)의 장단점을 비교합니다.
- **용어:** 헛갈리기 쉬운 **Epoch(에폭)**과 **Iteration(반복)**의 개념을 명확히 합니다.
- **원리:** 왜 배치 크기를 2^n (**64, 128** 등)으로 설정해야 하드웨어(CPU/GPU)가 좋아 하는지 배웁니다.
- **구현:** 데이터를 무작위로 섞고(Shuffle) 나누는(Partition) 코드를 작성합니다.

2 Essential Terminology

용어	설명	예시 ($m = 1000$, Batch=100)
Epoch	전체 데이터(m)를 한 번 다 훑는 것.	책 1권을 1회독 함.
Iteration	파라미터를 한 번 업데이트하는 것.	문제 100개를 풀고 채점함.
Mini-batch	한 번의 Iteration에 쓰이는 데이터 묶음.	1 Epoch = 10 Iterations.

3 Core Concepts: 학습 방식의 스펙트럼

3.1 1. The Three Types of Gradient Descent

데이터셋 크기가 m 일 때, 한 번의 업데이트에 몇 개의 데이터를 쓰는가?

- **Batch Gradient Descent (BGD):**
 - 데이터: 전체 m 개 사용.
 - 특징: 안정적이지만 너무 느림. 메모리 부족 위험.
- **Stochastic Gradient Descent (SGD):**
 - 데이터: 딱 1개 사용.
 - 특징: 엄청 빠르지만 벡터화(병렬 처리) 이점이 없음. 진동이 심함.
- **Mini-batch Gradient Descent:**
 - 데이터: T 개 사용 (예: 64, 128).
 - 특징: **Sweet Spot**. BGD의 안정성 + SGD의 속도 + 벡터화 효율성을 모두 잡음.

□ 산 내려오기 비유 (직관적 비유)

- **Batch:** 지도를 펼쳐 산 전체를 파악한 뒤, 정확하게 한 발자국 내딛습니다. (너무 신중함)
- **Stochastic:** 눈을 감고 발끝 감각만으로 미친 듯이 뛰어내려 갑니다. (빠르지만 비틀거림)
- **Mini-batch:** 100 걸음 앞만 보고 방향을 잡아 내려갑니다. (적당히 빠르고 적당히 정확함)

3.2 2. Why Powers of 2? (2^n)

”교수님, 배치 크기를 100개나 50개로 하면 안 되나요?”

- 됩니다. 하지만 **비효율적**입니다.
- 컴퓨터 메모리(CPU/GPU)의 주소 체계와 캐시는 **2진수 기반**입니다.
- 32, 64, 128, 256, 512 등으로 설정하면 메모리 정렬(Alignment)이 딱 맞아떨어져 연산 속도가 최적화됩니다.

4 Implementation: Shuffle and Partition

데이터를 섞고(Shuffle) 자르는(Partition) 과정을 구현합니다. 가장 중요한 점은 X 와 Y 를 똑같은 순서로 섞어야 한다는 것입니다.

```
1 import numpy as np
2 import math
3
4 def random_mini_batches(X, Y, mini_batch_size=64, seed=0):
5     """
6     X: (n_x, m), Y: (1, m)
7     """
8     np.random.seed(seed)
9     m = X.shape[1] # 데이터 샘플 수
10    mini_batches = []
11
12    # Step 1: Shuffle (X, Y)
13    # 0 ~ m-1 까지의 무작위 인덱스 생성
14    permutation = list(np.random.permutation(m))
15
16    # 중요: 와X 를Y 같은 인덱스로 섞음 열( 단위)
17    shuffled_X = X[:, permutation]
18    shuffled_Y = Y[:, permutation].reshape((1, m))
19
20    # Step 2: Partition 나누기()
21    # 짝 찬 배치의 개수
22    num_complete_minibatches = math.floor(m / mini_batch_size)
23
24    for k in range(0, num_complete_minibatches):
25        begin = k * mini_batch_size
26        end = (k + 1) * mini_batch_size
27
28        mini_batch_X = shuffled_X[:, begin : end]
29        mini_batch_Y = shuffled_Y[:, begin : end]
30        mini_batches.append((mini_batch_X, mini_batch_Y))
31
32    # Step 3: Handling the Remainder 자투리( 처리)
33    # 데이터가 딱 나누어떨어지지 않을 때 마지막 배치를 처리
34    if m % mini_batch_size != 0:
35        begin = num_complete_minibatches * mini_batch_size
36        # 끝까지 다 담기
37        mini_batch_X = shuffled_X[:, begin : ]
38        mini_batch_Y = shuffled_Y[:, begin : ]
39        mini_batches.append((mini_batch_X, mini_batch_Y))
40
41    return mini_batches
```

Listing 1: Random Mini-batches Generator

5 FAQ & Pitfalls

Shuffle 시 주의사항 (오해 방지 가이드)

시계열 데이터(주식, 날씨, 음성 등)처럼 순서(Time)가 중요한 데이터는 절대 섞으면 안 됩니다! 과거 데이터로 미래를 예측해야 하는데, 섞어버리면 미래를 보고 과거를 맞추는 꼴(Data Leakage)이 됩니다.

Q. 배치 크기가 너무 크면(8192 이상) 어떻게 되나요?

A. GPU 메모리 부족(OOM Error)이 발생할 수 있습니다. 또한, 모델이 너무 일반화된 패턴만

배워서 성능(Generalization)이 떨어지는 현상(Sharp Minima)이 발생할 수 있습니다. 보통 32 512 사이를 권장합니다.

다음 단계 (Next Step)

미니 배치를 쓰니 학습이 빨라졌습니다. 하지만 비용 함수 그래프를 확대해 보면 여전히 지그재그로 진동하며 내려갑니다.

이 진동을 줄이고, 내리막길에서 공이 굴러가듯 **관성(Inertia)**을 붙여 더 빠르게 내려가게 할 수는 없을까요? 다음 시간에는 단순한 경사 하강법을 넘어선 **[Optimization] Momentum (모멘텀)** 알고리즘에 대해 배우겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Mini-batch GD:** 데이터를 작은 묶음으로 나누어 업데이트한다. (속도 + 안정성)
2. **Power of 2:** 배치 크기는 2^n (32, 64, 128...) 이 하드웨어 효율적이다.
3. **Shuffle:** 매 에폭마다 데이터를 섞어주어야 학습이 골고루 된다.
4. **Last Batch:** 데이터가 나누어떨어지지 않을 때, 마지막 자투리 배치를 버리지 말고 처리해야 한다.